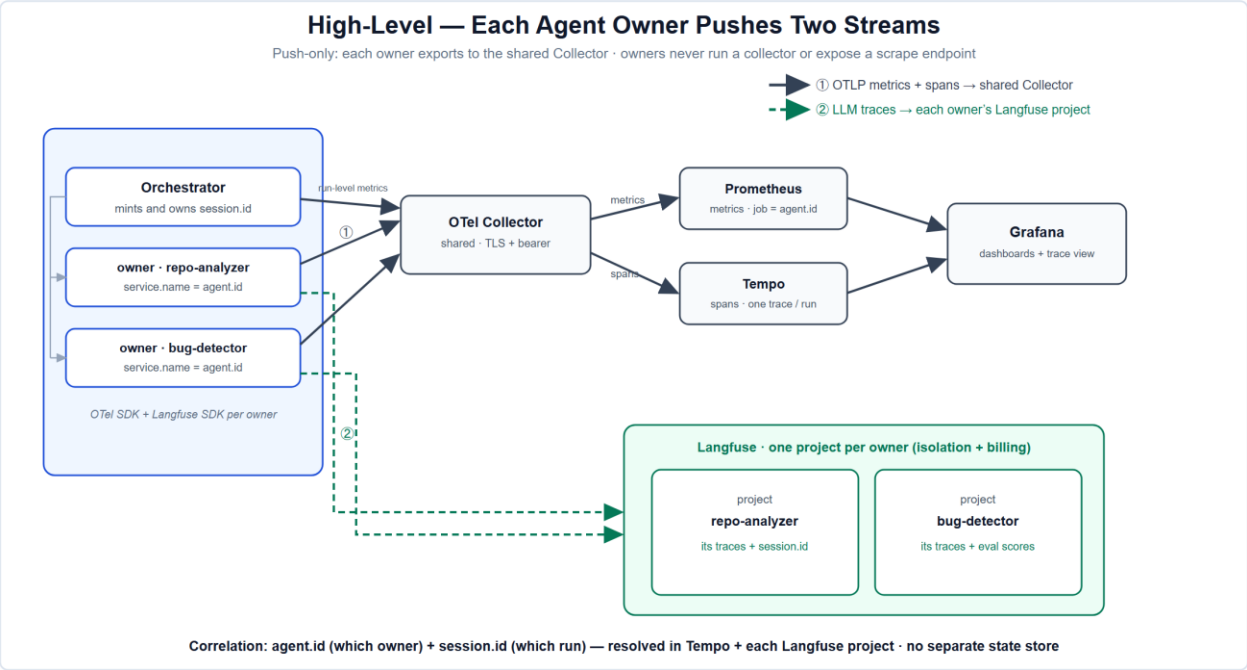
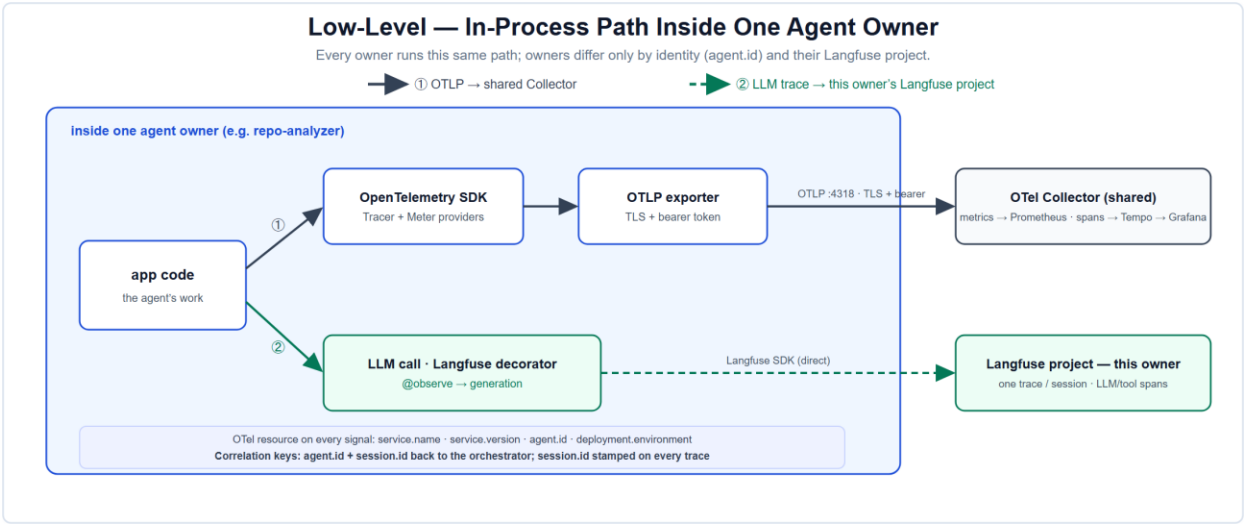
Observability Integration Contract

1 orchestrator | 2 owners: repo-analyzer + bug-detector | 2 streams | 3 backends: Prometheus, Tempo, Grafana

Architecture





Mandatory on every signal: service.name, service.version, agent.id, deployment.environment (OTel resource, from OTEL_RESOURCE_ATTRIBUTES) and session.id (minted by the orchestrator, on the owner's trace). agent.id + session.id are the correlation keys: agent.id identifies the owner, session.id joins one run across owners.

This simplified deployment has no Session Manager/PostgreSQL; session.id is resolved through Tempo and Langfuse.

Section 1 - Onboarding & Connectivity Requirements

Every item below is mandatory before an agent owner (repo-analyzer, bug-detector) is considered onboarded.

1. **Credentials.** Obtain the shared Collector's OTLP bearer token (OTLP_BEARER_TOKEN; presented via OTEL_EXPORTER_OTLP_HEADERS) and your own Langfuse project keys. TLS is deploy-time: the same env vars switch to https:// plus the platform certificate with no code change.
2. **SDKs.** Install opentelemetry-sdk, opentelemetry-exporter-otlp-proto-http, and langfuse.
3. **Identity.** service.name = agent.id = your owner id (repo-analyzer or bug-detector), plus service.version and deployment.environment. One owner equals one identity across metrics, spans, and its Langfuse project.

4. **Instrument your run.** We hand you a session.id per run; wrap your work in propagate_attributes(session_id=..., environment=...) and wrap the run root in @observe(). LLM calls use @observe(as_type="generation"). One trace per session in your own project, every trace carrying agent.id + session.id.
5. **Verification.** Onboarded only when both appear on our side: for example, agent.invocations{agent.id="bug-detector"} on Grafana/Prometheus and its run trace (generations + session.id) in the bug-detector Langfuse project.

Section 2 - Telemetry Compliance Requirements

Owners must emit these baseline metrics, each labelled agent.id:

Metric (dot-style)	Type	Labels	Meaning
agent.invocations	counter	agent.id	Request count
agent.duration	histogram	agent.id	Per-invocation latency (seconds)
agent.errors	counter	agent.id, error.type	Invocations that raised
gen_ai.client.token.usage	histogram	agent.id, gen_ai.token.type, gen_ai.request.model	LLM tokens
llm.cost.usd	counter	agent.id, gen_ai.request.model	Estimated LLM spend (USD)

The orchestrator emits sessions.active, runs.total, and run.duration. Owners emit only the per-invocation metrics above.

Self-service extras. Optional metrics include gen_ai.client.operation.duration, gen_ai.server.time_to_first_token, gen_ai.finish_reasons, and the bug-detector's eval.bug_recall / eval.judge_quality (0..1).

Langfuse rules. Each owner emits to its own Langfuse project for isolation and billing. Each trace MUST include agent.id and session.id metadata so traces can be correlated centrally. Emit one trace per session, with spans for each LLM or tool call.

Long-running owners MUST emit service.heartbeat; run-once jobs MUST flush metrics/traces before exit.

Reference code

```
# env (your agent owner)
#   OTEL_EXPORTER_OTLP_ENDPOINT = https://collector.us/        (we provide it; TLS)
#   OTEL_EXPORTER_OTLP_HEADERS  = Authorization=Bearer%20<OTLP_BEARER_TOKEN>
#   OTEL_RESOURCE_ATTRIBUTES    = service.name=<your-agent>,service.version=1.0.0,
#                                 agent.id=<your-agent>,deployment.environment=prod
#   LANGFUSE_HOST / LANGFUSE_PUBLIC_KEY / LANGFUSE_SECRET_KEY  (your own project keys)

import time
from opentelemetry import metrics, trace
from opentelemetry.sdk.metrics import MeterProvider
from opentelemetry.sdk.metrics.export import PeriodicExportingMetricReader
from opentelemetry.exporter.otlp.proto.http.metric_exporter import OTLPMetricExporter
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor
from opentelemetry.exporter.otlp.proto.http.trace_exporter import OTLPSpanExporter
from opentelemetry.sdk.resources import Resource
from langfuse import Langfuse, observe, get_client, propagate_attributes
from your_app import provider  # provider.chat(model, prompt) -> reply(.text, .in_tokens, .out_tokens)

AGENT_ID = "your-agent"                       # = service.name = agent.id
L = {"agent.id": AGENT_ID}                    # every metric carries agent.id
PRICE_IN, PRICE_OUT = 0.05 / 1e6, 0.20 / 1e6   # example USD per input/output token

metrics.set_meter_provider(MeterProvider(resource=Resource.create(),
    metric_readers=[PeriodicExportingMetricReader(OTLPMetricExporter(), export_interval_millis=5000)]))
m = metrics.get_meter(AGENT_ID)

tp = TracerProvider(resource=Resource.create())
tp.add_span_processor(BatchSpanProcessor(OTLPSpanExporter()))
trace.set_tracer_provider(tp)
tracer = trace.get_tracer(f"{AGENT_ID}.tracer")

invocations = m.create_counter("agent.invocations")
duration    = m.create_histogram("agent.duration", unit="s")
errors      = m.create_counter("agent.errors")
tokens      = m.create_histogram("gen_ai.client.token.usage", unit="{token}")
cost        = m.create_counter("llm.cost.usd")

Langfuse(blocked_instrumentation_scopes=[f"{AGENT_ID}.tracer"])

@observe(name="agent.run")
def run(task, *, session_id):
```

```
    with propagate_attributes(session_id=session_id, environment="prod", metadata=L), \
         tracer.start_as_current_span("agent.run"):
        invocations.add(1, L)
        t0 = time.perf_counter()
        try:
            return call_llm(task, model="gpt-oss:20b")
        except Exception as e:
            errors.add(1, {**L, "error.type": type(e).__name__}); raise
        finally:
            duration.record(time.perf_counter() - t0, L)

@observe(name="llm.generate", as_type="generation")
def call_llm(prompt, *, model):
    reply = provider.chat(model=model, prompt=prompt)
    ml = {**L, "gen_ai.request.model": model}
    tokens.record(reply.in_tokens,  {**ml, "gen_ai.token.type": "input"})
    tokens.record(reply.out_tokens, {**ml, "gen_ai.token.type": "output"})
    cost.add(reply.in_tokens * PRICE_IN + reply.out_tokens * PRICE_OUT, ml)
    get_client().update_current_generation(
        model=model, usage_details={"input": reply.in_tokens, "output": reply.out_tokens})
    return reply.text
```

Guidelines

- Dot-style, stable metric names: <domain>.<metric> (agent.duration, gen_ai.*, eval.*). Renames are breaking changes.
- Always include agent.id + session.id: agent.id on every metric and trace, session.id on every trace. These are the correlation keys back to the orchestrator.
- No unbounded / high-cardinality label values: no raw run IDs, emails, or URLs in metric labels. session.id rides on traces, not on metrics.
- Never put secrets or PII in labels, span names, or Langfuse payloads. Content capture is gated by OTEL_INSTRUMENTATION_GENAI_CAPTURE_MESSAGE_CONTENT=false for metadata-only mode.
- Always set service.version; regressions are tracked across releases.
- New mandatory metrics are announced by the platform in this contract first; optional metrics are self-service if they follow these conventions.